

CASA and cult-cargo

Landman Bester

SARAO

2026-03-25

Stimela Recap — Cabs

- **Cabs** are atomic units of work — like functions
- Each cab has a well-defined **schema**: typed inputs and outputs
- A cab does one thing and does it well

Cab = function with a schema

Inputs → Cab → Outputs

Stimela Recap — Recipes

- **Recipes** also have a schema (inputs and outputs) — but they are more like scripts than functions
- A recipe strings multiple cabs together as a sequence of **steps**
- This sequence of steps defines a **pipeline**

Recipe = script that chains cabs into a pipeline

Step 1 (Cab A) → Step 2 (Cab B) → Step 3 (Cab C)

The `_include` Construct

`_include` lives at the top of a recipe and **merges** external YAML into it — bringing in cab definitions from CultCargo or your own files.

```yaml
_include:
  - (cultcargo)wsclean.yml    # curated cab from cult-cargo
  - my-cabs.yml               # user-defined cabs
```

The `(module)file.yml` syntax resolves `file.yml` from inside an installed Python package — so `(cultcargo)wsclean.yml` looks for `wsclean.yml` inside the `cultcargo` package. The file must exist in that package or the include fails.

> Think of it like Python's `dict.update` — keys from included files are merged in, and you can override any leaf value.

`_include` as a Merge

```
# _include merges like dict.update
base = {"cabs": {"wsclean": {"command": "wsclean"}}}
override = {"cabs": {"wsclean": {"command": "~/my/wsclean"}}}

base["cabs"]["wsclean"].update(override["cabs"]["wsclean"])
# => {"cabs": {"wsclean": {"command": "~/my/wsclean"}}}
```

Stimela equivalent:

```
_include:
  - (cultcargo)wsclean.yml

cabs:
  wsclean:
    command: ~/my/wsclean
```

Subsections **merge**, leaf values **overwrite** — just like nested `dict.update`.

Cab Flavours

Cabs come in four flavours, set by the `flavour` field:

Flavour	`command` is...	Example
`binary`	an executable	`/bin/cp`
`python`	a callable	`os.getloadavg`
`python-code`	inline code	`c = a + b`
`casa-task`	a CASA task	`flagdata`

```yaml
cabs:
  cp:
    flavour: binary
    command: /bin/cp
```

Binary — runs an executable

```yaml
cabs:
  add:
    flavour: python
    command: module.function
```

Python — calls a python function

CASA5 vs CASA6 Cabs

CASA5 — monolithic

- Shipped its own Python 2.7 + IPython shell
- Tasks only available inside the CASA interpreter
- Stimela flavour: `casa-task` — launches an external CASA binary to run the task

CASA6 — modular

- `pip install casatools casatasks` into any Python 3 environment
- Tasks are regular Python callables
- Stimela flavour: `python` — just imports and calls the function directly
- Proper exception handling

`cult-cargo` only ships CASA6-style (`python` flavour) cabs. You would need to write your own if you wanted to use CASA5.

Discovering CASA Cabs in cult-cargo

List all available CASA cabs (note `-l` flag):

```
stimela doc -l cultcargo.casa::
```

Get help on a specific cab:

```
stimela doc cultcargo.casa:: casa.bandpass
```

`cultcargo.casa::` scopes the query to the `casa` module in the `cultcargo` package. Get help on a specific cab by dropping `-l` and adding appending the cab name (in this case `casa.bandpass`).

Example - Getting the Data

Download the data available here to get started.

Extract the data using

```
mkdir data
tar -xvf ARIWS.INTROFIMAGING.TriA.1559937657.tar.gz -C data/
ls -lrt data/
1559937657.PRIMARY.1.0.1.1.ghz.avg832khz32s.12ants.scan2.3.ms  # pr
1559937657.SECONDARY.1.0.1.1.ghz.avg832khz32s.12ants.ms  # secondar
1559937657.RXCJ1638.2-6420.1.0.1.1.ghz.avg832khz12s.12ants.scan2.ms
```

Example - Building the Recipe

1. `listobs` — list the details of all observations
2. `flagdata.save` — backup the FLAG column in each MS
3. `flagsummary` — report how much data are flagged in each MS
4. `plotms` — inspect off-diagonal correlations: amplitude vs frequency for baselines 1–100 m
5. `flagdata` — flag corrupted data
6. **KGB calibration** — solve for gains on the primary calibrator field
7. **Solution transfer** — transfer solutions to secondary
8. **Phase calibration** — solve for the phases on the secondary calibrator field
9. **Quality assurance** — make QA plots to ensure the calibration was successful
10. **Image + selfcal** — image and self-calibrate the target field

Updating cult-cargo

Before we begin, make sure you have the latest version:

```
pip install -U 'cult-cargo==0.2.1rc6'
```

> This release includes bug fixes and new cabs you'll need for your assignment.

User Cabs + Curated Cabs

```
_include:
  - (cultcargo.casa)gaincal.yml   # curated cab

cabs:  # user cabs
  read_ms:
    flavour: python-code
    command: |
    from pyrap.tables import table  # or use casatools

    ms = table(ms_path)
    uvw = table.getcols('UVW')
    ant1 = table.getcols('ANTENNA1')
    ant2 = table.getcols('ANTENNA2')

    inputs:
      ms_path:
        dtype: MS
        required: true
```

Running Containers Locally

You can drop into the container directly e.g.

```
apptainer shell --bind /home/bester/NASSP/TriA_example:/TriA_example:rw \
             ~/.singularity/quay.io-stimela2-python-astro:cc0.2.0.simg
```

where `--bind <host_path>:<mount_path>[:rw]` mounts a directory as read-write inside the conatiner (i.e. persistent changes).

> This is useful for debugging and checking which packages are available in a container. You should be able to install a new package this way if you need to.

CASA plotms — Visibility Inspection

The `plotms` tool is useful for plotting visibilities and/or gains. For example, plot amplitude against UV distance for specific anetnnas:

```yaml
steps:
  plot_amp_vs_uvdist:
    cab: casa.plotms
    params:
      ms: =recipe.ms_path
      xaxis: uvdist
      yaxis: amp
      ydatacolumn: data
      coloraxis: corr
      selectdata: true
      antenna: '0&1'    # antenna selection!
      plotrange: [0, 0, 0, 0]
      plotfile: '{recipe.output_dir}/data_amp_vs_uvdist.png'
      overwrite: true
```

`showgui` is not exposed, the plot is saved to `plotfile`.

CASA gaincal — Solving for Gains

`gaincal` solves for antenna-based complex gains e.g. amplitude calibration:

```yaml
steps:
  ampcal:
    cab: casa.gaincal
    params:
      ms: =recipe.ms
      refant: =recipe.refant  # be consistent
      refantmode: "strict"    # force consistency
      calmode: a              # "a" for amplitude
      caltable: '{recipe.output_dir}/gcal.G0a'
      solint: "100s"          # cadence
      minsnr: 1e-6            # avoid flagging solutions
```

`stimela doc culcargo.casa:: casa.gaincal` (check `See also` link).

Plotting gains with `plotms`

For example, to plot gain amplitude vs. time:

```
steps:
  plotgainsant:
    cab: casa.plotms
    params:
      ms: =steps.ampcal.caltable   # pass gain table as `ms`
      xaxis: "ant1"                # plot ANTENNA1 on the x-axis
      yaxis: "GainAmp"             # Gain amplitude on the y-axis
      plotfile: '{recipe.output_dir}/plotms_gains_vs_ant.png'
      overwrite: true
```

Note the reference to output of the the `ampcal` step. This is useful for structuring recipes (and avoiding copy paste errors).

Phase calibration with amplitude solutions

Sometimes we need to apply previously solved solutions while calibrating for another term (e.g. phase calibration with different solution interval A.K.A. cadence):

```yaml
steps:
  phasecal:
    cab: casa.gaincal
    params:
      ms: =recipe.ms
      refant: =recipe.refant
      refantmode: "strict"
      calmode: "p"                                  # "p" for phase
      caltable: '{recipe.output_dir}/phasecal.G0p'  # gain to solve
      gaintable: [=steps.ampcal.caltable]           # list of gains to apply
      solint: '10s'                                 # different cadence
      minsnr: 1e-6
```

Applying Calibration Solutions

Once gains are solved, apply them with `applycal`:

```
steps:
  apply_gains:
    cab: casa.applycal
    params:
      ms: =recipe.ms
      gaintable: [=steps.ampcal.caltable, =steps.phasecal.caltable]
      interp: ['linear', 'nearest']  # defaults are usually fine
```

`gaintable` accepts a list — you can stack multiple calibration tables (bandpass, delay, gain) and apply them all at once. Useful to transfer calibration solutions and form corrected data for imaging.

Cell and Image Size for Imaging

The angular resolution of an interferometer is approximately:

$$\theta_{\mathrm{res}} \approx \frac{\lambda}{B_{\mathrm{max}}}$$

where B_{max} is the longest baseline and λ is the observing wavelength.

- Cell size $\approx \theta_{\mathrm{res}}/4$ is a good rule of thumb
- Example: at 1.4 GHz ($\lambda \approx 0.21$ m) with $B_{\mathrm{max}} = 8$ km:

$$\theta_{\mathrm{res}} \approx \frac{0.21}{8000} \approx 5.4'' \quad \Rightarrow \quad \mathrm{cell} \approx 1.3''$$

- Number of pixels set by required field of view

Basic wsclean Usage

`wsclean` is a common imager for radio interferometry. A minimal call:

```
steps:
  image:
    cab: wsclean
    params:
      ms: =recipe.ms
      prefix: '{recipe.output_dir}/image'
      size: 1024          # number of pixels 1024 x 1024
      scale: 1.3asec      # cell size in arc-seconds
      channels-out: 1     # number of output channels
      niter: 2000         # number of clean iterations
      gain: 0.1           # minor-loop gain
      mgain: 0.95         # major-cycle gain
      auto-mask: 5        # mask from 5-sigma
```

There are many-many options. Make sure to familiarise yourself with the common ones (e.g. how could you control the time cadence during imaging?)

Opening FITS Images with astropy

FITS files produced by wsclean can be viewed with `astropy`:

```
from astropy.io import fits
from astropy.wcs import WCS
import matplotlib.pyplot as plt

hdu = fits.open(f"{fits_file}")[0]
wcs = WCS(hdu.header, naxis=2)  # extract 2D WCS

fig, ax = plt.subplots(subplot_kw={'projection': wcs})
im = ax.imshow(hdu.data[0, 0],  # select Stokes I, freq 0
               origin='lower', cmap='inferno')
ax.set_xlabel('Right Ascension')
ax.set_ylabel('Declination')
plt.colorbar(im, label='Jy/beam')
plt.savefig(f"{png_file}", dpi=150, bbox_inches='tight')
```

How would you put this in a cab?

Questions!

lbester@sarao.ac.za